

Homework 1

Domain: Clinical Trial Listing

Personal Configuration

Value	Result
SID4	3209
PORT_BASE	8509 (8000 + 3209 mod 900)
PREFIX	s3209
SEED	3209
VERIFY_SEED	263209
DOMAIN_ID	1 -Clinical trial listings

Environment

Hardware	Microsoft Surface Laptop 3 — AMD Ryzen 5 (4 cores), integrated AMD Radeon (no GPU acceleration)
RAM	16GB
OS	Windows 11 Home 64-bit
Local model:	qwen3:8b served through Ollama 0.33.0, CPU-only inference, 4096-token context.

Part1

a) HTML

This is the application for the clinical trial listing submission form. This is the person who posts the study to the public catalogue.

Entity schema

Field	Type
Trial Title	text
Sponsor / Institution	text
Submitter Email	email
Trial Summary	textarea
Status	select
Terms	checkbox

The screenshot shows a web browser at localhost:8509 displaying a 'Clinical Trial Listing' form. The form fields are: Trial Title (Phase II Study of Drug X in Diabetes), Sponsor / Institution (Stanford Medical Center), Submitter Email (test@hospital.org), Trial Summary (This study tests a new diabetes drug in adult patients over six months.), Status (Recruiting), and a checked checkbox for 'I agree to the terms and conditions.' A 'Submit Trial Listing' button is at the bottom.

The browser's console shows a failed resource load for a status of 404 (Not Found). The form data is converted to a JSON string and logged. The console output includes:

```
Failed to load resource: the server responded with a status of 404 (Not Found)
form data as JSON string: [{"trialTitle":"Phase II Study of Drug X add.js:148
in Diabetes","sponsor":"Stanford Medical Center","email":"test@hospital.org","content":"This study tests a new diabetes drug in adult patients over six months.","status":"Recruiting"}]
Trial Title: Phase II Study of Drug X in Diabetes add.js:153
Email: test@hospital.org add.js:154
Updated object with submissionDate: add.js:158
[{"trialTitle": "Phase II Study of Drug X in Diabetes", "sponsor": "Stanford Medical Center", "email": "test@hospital.org", "content": "This study tests a new diabetes drug in adult patients over six months.", "status": "Recruiting"}]
Successful submission count: 1 add.js:162
```

b) JavaScript

I have used JavaScript for validation. An arrow function is used to validate the length of the summary and the terms box is checked. In case of failure, it shows an alert. On a valid submission the form data is converted to a JSON string and logged, and Object destructuring extracts the trial title and email. A closure maintains a private counter that increments and logs each successful submission.

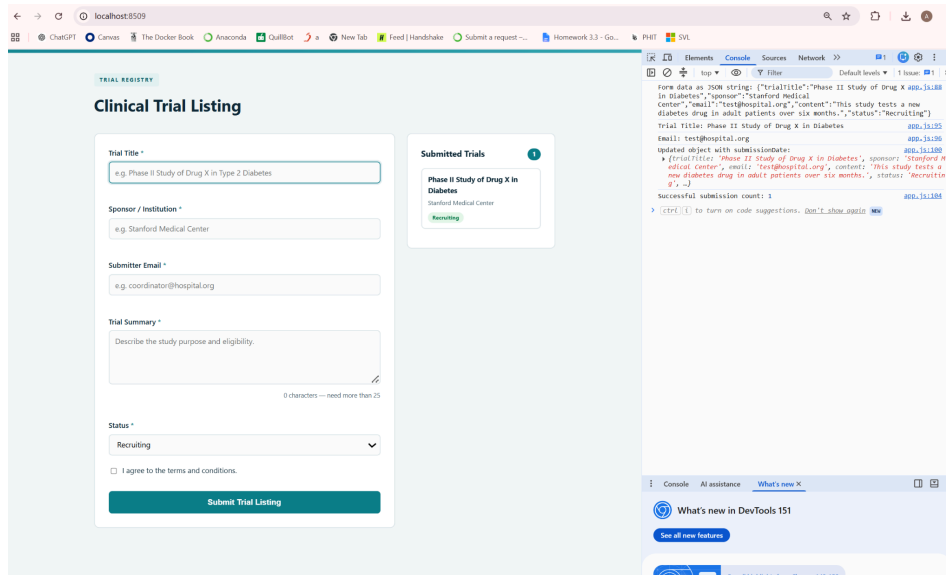
Deployment- Docker(Local)

I have created the custom Nginx config to override the default port 80 because the app must run on `PORT_BASE = 8509`.

Command:

`docker build -t clinical-trial-app.`

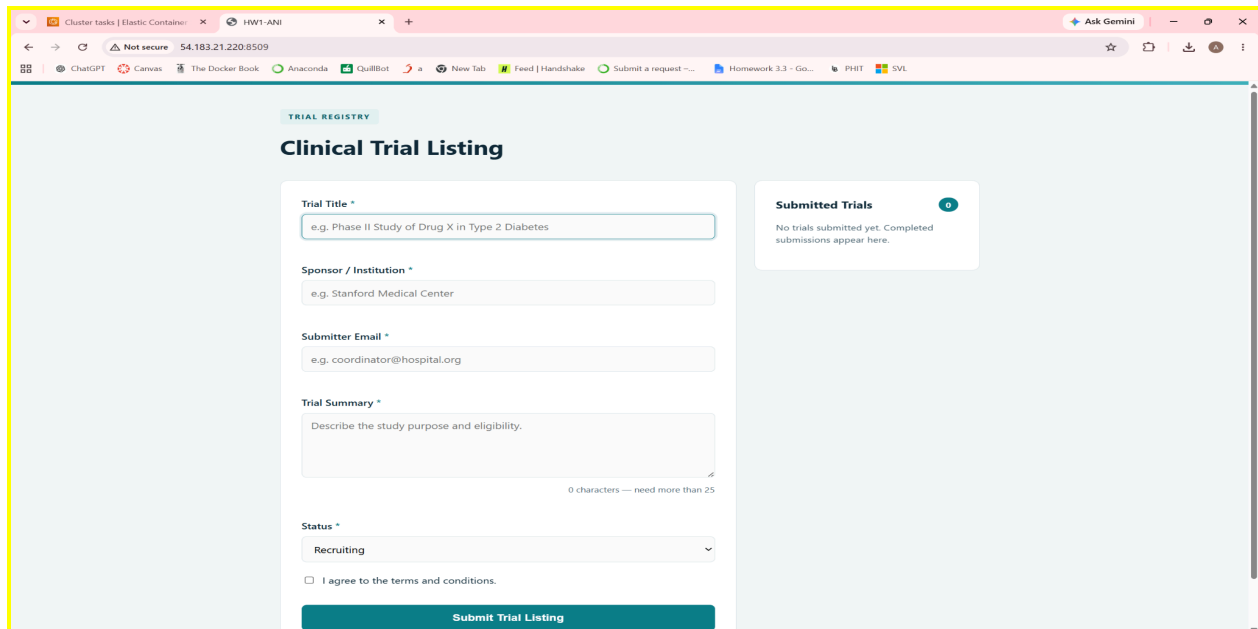
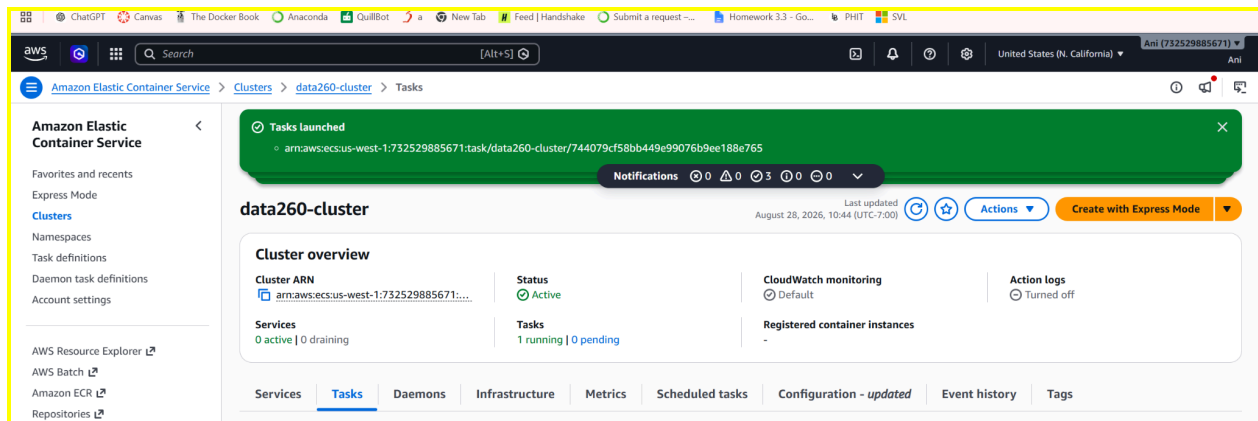
`docker run -d -p 8509:8509 clinical-trial-app`



Note: First I have created the normal form then I have added style with the help of AI to make it more interactive.

Deployment - AWS ECS

The same Docker image is run as a single ECS task First The image is pushed to Amazon ECR, an ECS service with desired count 1 runs it, and the task's security group opens inbound TCP on port 8509 so the public IP is reachable.



Part2

Agentic AI

There are two agents and one finalization step processes a title and content and return exactly three tags and a summary of at most 25 words as structured JSON.

Planner- Proposes tags and a summary

Reviewer - Checks and revises them

Finalizer - It emits the final result.

Command

py -3.12 agents_demo.py --title "Diabetes Drug Trial" \

--content "A six month randomized trial testing a new oral medication for type 2 diabetes in adults."

```
--- Planner (382819 ms) ---
{
  "thought": "Identifying key elements from the title and content to generate relevant tags and summary.",
  "message": "The task requires extracting three multi-word tags and a concise summary from the provided title and content about a diabetes drug trial.",
  "data": {
    "tags": [
      "randomized clinical trial",
      "oral medication for diabetes",
      "type 2 diabetes treatment"
    ],
    "summary": "Six-month randomized trial tests new oral medication for type 2 diabetes in adults.",
    "issues": []
  },
  "_tokens": {
    "input": 459,
    "output": 117,
    "total": 576
  }
}
```

```
--- Reviewer (364761 ms) ---
{
  "thought": "Extracting specific tags and a concise summary from the trial description.",
  "message": "Tags and summary derived from trial details. Summary captures key elements within word limit. No data issues detected.",
  "data": {
    "tags": [
      "type 2 diabetes treatment",
      "oral medication trials",
      "randomized controlled trials"
    ],
    "summary": "Six-month trial tests new oral medication for type 2 diabetes in adults.",
    "issues": []
  },
  "_tokens": {
    "input": 506,
    "output": 103,
    "total": 609
  }
}
```

```
=== Finalized Output ===
{
  "thought": "Extracting relevant tags and summary based on trial specifics and constraints.",
  "message": "Tags and summary focus on trial type, condition, and treatment. Summary adheres to word limit. No issues detected.",
  "data": {
    "tags": [
      "type 2 diabetes treatment",
      "randomized clinical trial",
      "oral medication trial"
    ],
    "summary": "A six-month trial tests a new oral medication for type 2 diabetes management.",
    "issues": []
  },
  "_tokens": {
    "input": 439,
    "output": 107,
    "total": 546
  }
}

=== Publish Package ===
{
  "title": "Diabetes Drug Trial",
  "email": "student@example.com",
  "content": "A six month randomized trial testing a new oral medication for type 2 diabetes in adults.",
  "agents": {
    "transcript": [
      {
        "role": "Planner",
        "content": "The task requires extracting three multi-word tags and a concise summary from the provided title and content about a diabetes drug trial."
      },
      {
        "role": "Reviewer",
        "content": "Tags and summary derived from trial details. Summary captures key elements within word limit. No data issues detected."
      }
    ],
    "final": {
      "tags": [
        "type 2 diabetes treatment",
        "randomized clinical trial",
        "oral medication trial"
      ],
      "summary": "A six-month trial tests a new oral medication for type 2 diabetes management.",
      "issues": []
    }
  },
  "submissionDate": "2026-08-26T22:11:00Z"
}
```

Short Answer

1) Final Tags

type 2 diabetes treatment, randomized clinical trial, oral medication trial

2) Final summary

A six-month trial tests a new oral medication for type 2 diabetes management.

3) Did the Reviewer change anything?

Yes. The Reviewer reordered two of the Planner's tags for example "oral medication for diabetes" became "oral medication trials" and "randomized clinical trial" became "randomized controlled trials" and tightened the summary, even though it reported no formal issues. The Finalizer then settled on a blend of the two.

Explanation:

Input: I pass a trial title and content on the command line. The agents only see this text so the tags come from whatever input is given.

Planner: The first agent reads the title and content and proposes three topical tags and a short summary which should be greater than twenty-five words and returning them as JSON.

Reviewer: The second agent receives the Planner's result and checks its whether the tags are specific and topical, whether the summary is within the word limit and whether there's any unwanted formatting. If needed, It can revise the tags or summary and list any problems it finds. In my run it reordered two of the Planner's tags.

Finalizer: The final step takes the reviewed output and produces the clean final result exactly three tags and the final summary with the issues list cleared. This is what gets printed as the final JSON.

Shared adapter: All three agents talk to the model through one function, `complete()`, in `src/model_client.py`. None of them call Ollama directly they go through this single adapter, so the model setup lives in one place.

Part3

Measuring Non- Determinism

As mentioned, Pipeline was run on one fixed input 20 times at temperature 0.0 and 20 times at temperature 0.7. For each run the final tags set and wall clock latency were recorded to [reports/hw01/raw/nondeterminism_runs.csv](#).

Results

Metric	Temp 0.0	Temp 0.7
Distinct tag sets	3	14
Tags in all 20 runs	oral medication, type 2 diabetes	none
Tags in exactly 1 run	none	10 tags (e.g. oral antidiabetic therapy, study phase, phase ii clinical trials)

Latency	Temp 0.0	Temp 0.7
p50(ms)	724452	898426
p95(ms)	928249	1448281
p99(ms)	932289	1456572

```

tags=['phase ii trials', 'type 2 diabetes', 'oral medication'] latency=87926ms
[temp 0.7] run 18/20 ...
tags=['phase ii clinical trials', 'type 2 diabetes', 'oral medications'] latency=897493ms
[temp 0.7] run 19/20 ...
tags=['phase ii trials', 'type 2 diabetes', 'oral medication'] latency=1037055ms
[temp 0.7] run 20/20 ...
tags=['phase ii trial', 'type 2 diabetes', 'blood glucose control'] latency=899358ms

All 40 runs complete.

===== PART 3 METRICS =====

--- Temp 0.0 (20 runs) ---
Distinct tag sets : 3
Tags in all runs : ['oral medication', 'type 2 diabetes']
Tags in exactly 1 : (none)
Latency p50/p95/p99 : 724451.5 / 928248.8 / 932289.0 ms

--- Temp 0.7 (20 runs) ---
Distinct tag sets : 14
Tags in all runs : (none)
Tags in exactly 1 : ['oral medication testing', 'oral antidiabetic therapy', 'oral medication for blood glucose', 'phase ii clinical trials', 'oral medication for diabetes', 'phase ii study', 'target condition', 'treatment method', 'study phase', 'phase ii clinical trials']
Latency p50/p95/p99 : 898425.5 / 1448281.4 / 1456572.3 ms

```

What two users with identical input see

Two users sending the identical input can get different results depending on temperature.

At temperature 0.0, the two users see nearly identical output. Across my 20 runs there were only 3 distinct tag sets, and two of the three tags type 2 diabetes and oral medication appeared in all 20 runs. The only variation was the middle tag drifting between near-synonyms ("phase ii trial" / "phase ii study" / "phase ii trials"). So two users would almost certainly agree on the core tags and might differ only in that one phrasing. Note this is *near*-deterministic, not guaranteed: temp 0.0 greatly reduces variation but did not eliminate it.

At temperature 0.7, the two users likely see different results. My 20 runs produced 14 distinct tag sets, no tag appeared in all 20 runs, and 10 tags appeared only once (e.g. "oral antidiabetic therapy", "study phase", "phase ii clinical trials", plus hyphenated variants like "type-2-diabetes"). Two users comparing notes would frequently see non-matching tags sometimes only a loose overlap on the underlying concepts.

Where run-to-run variation is acceptable: a brainstorming / tag-suggestion aid, where the user reviews and edits the output. If a researcher is drafting keywords for a trial listing and the tool proposes slightly different phrasings each time eg "oral medication" vs "oral antidiabetic therapy" that variety is *useful* it surfaces alternatives the user can pick from. The human is the final filter, so non-determinism costs nothing.

Where it is not acceptable:

In any automated or reproducible pipeline e.g. tags used as database keys, deduplication keys or filters that other systems depend on. If run 1 stores a trial under type-2-diabetes and run 2 under type 2 diabetes a later search or join silently misses records. Anything requiring auditability or exact reproducibility needs stable output, so temp 0.7 would be unacceptable there you'd pin temp 0.0 instead.

Note: Reasoning was therefore disabled by adding `/no_think` to the system prompt inside the adapter. Because all model calls route through this one adapter, the single change sped up every part. Even so, the Part 3 non-determinism experiment remained heavy: it runs the full pipeline (Planner + Reviewer + Finalizer = 3 model calls) 40 times, which took more than 5–6 hours total on CPU-only hardware.

Part4 - Model Client and Token Accounting

A reusable adapter (`src/model_client.py`) exposes `complete(messages, tools=None)` every model call in Parts 2, 3 and 4 goes through it. `hw1_client.py` is a command-line demo that loads `AGENT.md` as the system prompt, prints per-turn token counts, supports a `/stats` command, and prints cumulative totals on exit.

Five-turn run

Turn	Input tokens	Output tokens	Total
1	191	1057	1248
2	282	430	712
3	384	520	904
4	587	665	1252
5	801	588	1389

Input tokens climb every turn because the whole conversation is resent each time.

```

you> /stats

===== /stats =====
Turns completed           : 3
Cumulative input tokens   : 857
Cumulative output tokens  : 2007
Cumulative total tokens   : 2864
Serialized history length: 2721 chars
=====

you> |

```

	After turn 3	After turn 5
Turns	3	5
Cumulative input	857	2245
Cumulative output	2007	3260
Cumulative total	2864	5505
History length (chars)	2721	4251

```

you> /stats

===== /stats =====
Turns completed           : 5
Cumulative input tokens   : 2245
Cumulative output tokens  : 3260
Cumulative total tokens   : 5505
Serialized history length: 4251 chars
=====

you> |

```

```
you> exit
```

```
===== SESSION SUMMARY =====
```

```
Total turns : 5
```

```
Cumulative input tokens : 2245
```

```
Cumulative output tokens : 3260
```

```
Cumulative total tokens : 5505
```

```
=====
```

```
PS C:\Users\amrut\Downloads\Part2>
```

AGENT.md verification: every code-review turn returned strict bullet-only output, and the model correctly flagged the SQL-injection vulnerability in the `get_user()` example.

```

Model: qwen3:8b
you> Please review this function: def add(a,b): return a+b

assistant>
- The function lacks proper indentation for the return statement, causing a syntax error.
- The function does not handle non-numeric inputs, which could lead to TypeError.
- The function name 'add' is appropriate, but could be more descriptive if the context requires it.
- No error handling is implemented for cases where inputs are not compatible with addition.

[turn 1] input=191 output=1057 total=1248
you> Review this: def divide(a,b): return a/b

assistant>
- The function lacks error handling for division by zero, which will raise a ZeroDivisionError.
- No validation is performed for non-numeric inputs, leading to potential TypeErrors.
- The return statement correctly performs division, but could use parentheses for clarity if nested.
- The function does not handle cases where 'b' is zero or non-numeric, making it unreliable in edge scenarios.

[turn 2] input=282 output=430 total=712
you> What are common mistakes when reading a file in Python?

assistant>
- Forgetting to close the file after reading, leading to resource leaks or corruption.
- Not using the 'with' statement, which automatically handles file closure.
- Using incorrect modes ('r', 'w', 'a') that overwrite or truncate files unintentionally.
- Failing to handle exceptions like FileNotFoundError or PermissionError.
- Not specifying encoding parameters, causing decoding errors with non-UTF-8 files.
- Reading large files into memory all at once, leading to high memory usage.
- Not checking if the file exists before attempting to open it.
- Using 'read()' without limits, which can cause excessive memory consumption.
- Ignoring binary vs. text mode differences, leading to data corruption.
- Not handling cases where the file is empty or contains unexpected content.

[turn 3] input=384 output=520 total=904
you> /stats

===== /stats =====
Turns completed      : 3
Cumulative input tokens : 857
Cumulative output tokens : 2807
Cumulative total tokens : 2864
Serialized history length: 2721 chars
=====

you> Review: def get_user(id): return db.query("SELECT * FROM users WHERE id=" + id)

assistant>
- The function is vulnerable to SQL injection due to direct string concatenation of user input.
- No parameterization is used, allowing malicious input to alter the SQL query structure.
- Lack of error handling for database exceptions like 'DatabaseError' or 'OperationalError'.
- No input validation for the 'id' parameter, which should be numeric.
- The SQL query could return multiple rows if 'id' is not unique, leading to unexpected results.
- No use of prepared statements or safe query parameters ('%', '?', etc.) to prevent injection.
- The function does not handle cases where the database connection ('db') is closed or invalid.
- No explicit transaction handling or rollback mechanisms for potential database errors.
- The query lacks 'LIMIT 1' or similar clauses, risking multiple result returns.
- No logging or error messages for debugging purposes in case of failures.

[turn 4] input=587 output=665 total=1252
you> Summarize the top 3 issues you found across all the code so far.

assistant>
- **Security vulnerabilities** like SQL injection from concatenating user input without parameterization.
- **Lack of error handling** for edge cases (e.g., division by zero, file not found, invalid database queries).
- **Insufficient input validation** for parameters, leading to potential type errors or unexpected behavior.

[turn 5] input=801 output=588 total=1389
you> /stats

===== /stats =====
Turns completed      : 5
Cumulative input tokens : 2245
Cumulative output tokens : 3260
Cumulative total tokens : 5505
Serialized history length: 4251 chars
=====

you> exit

```

Conceptual Questions:

- 1) The prior conversation context is resent every turn because the model is stateless it only sees what we send in the current request. To continue the conversation, the whole history is resent this is the reason turn 5 could summarise code from turns 1 to4.
- 2) The system prompt sets the model's role and rules for the whole session (here, bullet-only code review) and persists across turns a user message is a single turn's request that the model answers directly.
- 3) The input token grows over conversation because Each turn appends the previous question and the model's answer to the history that is resent, so the input size compounds from 191 tokens on turn 1 to 801 on turn 5, with the serialized history growing from 2721 to 4251 characters.
- 4) The context window limits the growth means the maximum tokens the model accepts per request (4096 here). Once the accumulated history nears that limit, older messages must be dropped or summarised, so the context window is the hard ceiling.

Note

qwen3:8b is a reasoning ("thinking") model that by default emits a long internal reasoning block before every answer. On CPU-only hardware this made each turn extremely slow often 30+ minutes. This was identified by calling the model directly (`ollama run qwen3:8b "..."`), which printed a long reasoning block before even a one-word answer.